GPWC PYQ 2025
(End Sem Solution)

1a)

```
#include <iostream>
using namespace std;
int main () {
        int sum, digit, largest = 0;

        cin >>n;
        while (n>0);

            d = n%10;
            if (d > large)
                large = d;

            n = n/10;
      }
      cout << large
      return 0;
```

1 b)

```
# include <iostream>
using namespace std;
int* getRating (int n) {
        int *p = new int[n];
        for (int i=0; i<n; i++)
                cin >> p[i];
        return p; }
int main () {
```

```
    int n;
    cin>>n;
    int *r = getRating(n);
    for(int i=0; i<n; i++)
        cout << r[i] << " ";

    delete[] r;
    return 0;
}
```

1c)
```
#include <iostream>
using namespace std;

enum ProductCategory { Electronics, Clothing, Groceries };
float discount(ProductCategory c, float p)
    if (c == Electronics) return p-p*0.1
    if (c == Clothing) return p-p*0.2;
    return p-p*0.05;
}
int main() {
    int c; float p;
    cin>> c >> p
    cout << discount((ProductCategory)c,p);

    return 0;
}
```

2a)

```
sf:: Sprite sprite [6];
int active=0;
if (Keyboard::isKeyPressed (Keyboard::Down)) {
        active++;
        if (active >5)
            active = 0;
    }
```

2b)

```
RectangleShape timeBar;
timeBar.setSize (Vector2f (300, 20));
float timeLeft = 6;
timeBar.setSize (Vector2f (timeLeft * 50, 20));
```

2c)

```
int score =0
sf:: Text text
if (Keyboard::isKeyPressed (Keyboard::Up))
        { score++; }

text.setString (to_string (score));
```

3a)

```
CircleShape c(10);
c.setFillColor (Color::Green);
c.setPosition (10,10);
window.draw(c);
c.setPosition (1900, 10);
window.draw (c);
c.setPosition (10,1060);
window.draw (c);
```

```
c.setPosition(1900, 1060);
    window.draw(c);
c.setFillColor(Color::Red);
c.setScale(2,1);
c.setPosition(960, 540);
window.draw(c);
}
```

3b)
```
if (KeyBoard::isKeyPressed::Left))
        player.setPosition(300,500);
        player.setScale(1,1);
    if (KeyBoard::isKeyPressed::Right))
            player.setPosition(700,500)
            player.setScale(-1,1);
```

3c)
```
Sound Buffer buffer;
buffer.loadFromFile("death.wav");
Sound sound;
sound.setBuffer(buffer);
if (player.getGlobalBounds().intersects(Branch.getGlobalBounds()))
    sound.play();
```

4a)
```
class MyBat{
    int a,b,c,d;
public:
        void setData();
        void getData(){};
```

4b)
```
class Rect{
        RectangleShape r;
public:
        void draw();
        r.setSize(Vector2f(10,10));
        window.draw(r);
    }
};
```

4c)
```
class Bat {
        RectangleShape bat;
public:
        Bat(float x, float y) {
            bat.setSize(Vector2f(100,5))
            bat.setPosition(x,y); }
        void draw(){
                window.draw(bat);
            }; }
Bat b(100,260);
b.draw();
```

5a)
```
class Temperature {
        float c;
public:
        Temperature(float f) {
                c=(f-32) * 5/9; } };
```

5b)
```
class AreaCalculator {
public:
    int calculate(int s) {
        return s*s;
    }
    int calculate(int l, int b)
    {
        return l*b;
    }
};
```

5c)
```
#include <iostream>
using namespace std;
class Person {
protected:
    string name;
    int age;
public:
    Person(string n, int a) { name=n; age=a; }
void display() {
        cout << name << " " << age
    }
};
class Student : public Person{
    int roll;
public:
    Student(string n, int a, int r) : Person(n, a) {
        roll=r; }
};
```

6a)
```
Texture t;
t.loadFromFile("crosshair.png");
Sprite s(t);
window.setMouseCursorVisible(false);
s.setOrigin(16,16);
s.setPosition(Mouse::getPosition(window).x,
              Mouse::getPosition(window).y);
window.draw(s);
```

6b)
```
window.setView(mainView);
window.draw(spritePlayer);
window.setView(hudView);
window.draw(textScore);
```

6c)
```
enum GameState {GameOver, Playing};
Texture t1, t2;
t1.loadFromFile("gameLanding.png");
t2.loadFromFile("gamePlay.png");
Sprite s;
GameState g = GameOver;
if (Keyboard::isKeyPressed(Keyboard::X))
        g = Playing;
if (g == GameOver) s.setTexture(t1);
else s.setTexture(t2);
window.draw(s);
```

7a)
```
void spawn(View hudView){
        sprite.setPosition(hudView.getCenter());
}
```

7b)

```
void rotatePlayer(Vector2i m) {
    float angle = atan2(m.y - sprite.getPosition().y,
                        m.x - sprite.getPosition().x);
    sprite.setRotation(angle);
}
```

7c)

```
if (Keyboard::isKeyPressed(Keyboard::D))
    sprite.move(-5,0);
mainView.setCenter(sprite.getPosition());
```

8a)

```
float angle = atan2(y1-y2, x1-x2)
bloater.setRotation(angle);
```

8b)

```
if (x1>x2) x2++;
else x2--;
if (y1>y2) y2++;
else y2--;
```

8c)

```
Zombie z[5];
for (int i=0; i<5; i++)
    z[i].spawn(rand()%500, rand()%500);
```

9a)

```
Texture t;
t.loadFromFile("background sheet.png")
Sprite s;
s.setTexture(t);
s.setTextureRect(IntRect(50,0,50,50));
s.setPosition(100,100);
window.draw(s);
```

9b)

```
RectangleShape bullet(Vector2f(2,2));
bullet.setPosition(x1, y1);
if (x2>x1) bullet.move(1,0);
else bullet.move(-1,0);
if (y2>y1) bullet.move(0,1);
else bullet.move(0,-1);
```

9c)

```
VertexArray arena(Quads);
arena.resize((600/50)*(600/50)*4);
```

10a)

```
Text t;
t.setString("Enter to start")
t.setPosition(960, 540);
if (Keyboard::isKeyPressed(Keyboard::Enter))
        t.setString("Playing");

window.draw(t)
```

10b)

```
#include<iostream>
using namespace std;
class student{
    public:
        student(){
            cout<<"student";}};
class sports{
    public:
        sports(){
            cout<<"sports";
        };
```

```
class Result : public student, public sports {
    public:
        Result () {
            cout << "Result";
        }
};
```

10 c)

```
#include <iostream>
using namespace std;
class Complex {
    int real, imag;
public:
    Complex (int r, int i) {
        real = r;
        imag = i;
    }
    Complex operator + ( Complex c) {
        return Complex (real+c.real, imag+c.imag);
    }
};
```